

RAG + Agent 기반 Agentic Translation QA — 최종 결과 보고서

한국어→영어 번역 품질 자율 판단·개선 시스템 구현 및 Baseline 대비 평가 · 2026-08-21

1. 시스템 아키텍처와 Agent 워크플로우

한국어→영어 번역에서 도메인 용어와 문장 품질을 검수하는 로컬 Translation QA 워크플로우를 구현함. Python 3.11+ · FastAPI 기반이며 공개 로컬 모델만 사용하고 외부 유료 API 호출 없음. Agent 프레임워크는 결정적 Python controller로 직접 구성함.

한국어 입력 → Agent 문장 분석(도메인·핵심 용어 판별) → RAG 용어집 검색(exact-first hybrid) → Marian NMT 번역(RAG 컨텍스트 반영) → Agent 품질 판단(source coverage + LLM Judge) → 품질 미달 시 RAG 보강 후 재번역(최대 2회) → 최종 번역 + 품질 판단 로그

동일 Marian NMT를 공유하는 **Baseline / RAG / Agent / Agent+RAG** 4개 조건을 구현해 검색과 Agent 각각의 기여를 분리 측정함. 모든 attempt에 검색 hit, 후보 번역, 판단, 단계별 latency, stop reason을 trace로 기록함. Reference와 수동 label은 runtime에 전달하지 않고 오프라인 채점에서만 사용함.

필수 API 엔드포인트	역할
POST /translate/baseline	RAG/Agent 미적용 단순 NMT 번역과 trace 반환
POST /translate/agent-rag	Agent+RAG 적용 번역, 품질 판단, retry trace 반환
POST /benchmark	고정 평가셋에 대한 Baseline vs Proposed 모드별 지표와 artifact 생성

2. 프레임워크·모델 선정 이유와 한계

영역	선택과 이유	확인된 한계
번역	Helsinki-NLP/opus-mt-ko-en — 공개 한→영 NMT, CPU 로컬 실행 안정, 4개 조건의 공통 기준선으로 적합함	도메인 용어·주어·고유명사 누락 발생, prompt 방식 glossary 주입에 부적합함
Agent LLM	Ollama qwen3:1.7b — 16GB 메모리 환경에서 구동 가능한 소형 multilingual JSON 출력 모델	호출당 약 11초 latency, false PASS, JSON-error label 불안정, 자기평가 편향
임베딩	multilingual MiniLM — 경량이며 한국어·영어 동시 검색 가능함	소규모 literal glossary에서 vector noise 발생
벡터 검색	exact-first hybrid + in-memory cosine — exact 매칭 우선, vector는 보조로 사용함	대규모 영속 index·운영 확장성 부족
API/흐름	FastAPI-Pydantic + 결정적 Python controller — 재현성과 trace 확보에 유리함	인증·queue-checkpoint·분산 운영 미구현

파일럿 실험 근거: force_words_ids 강제 디코딩은 반복·퇴행 출력을, source augmentation은 용어 미반영을 보여 채택하지 않음. Exact retrieval은 용어 12/12를 찾았으나 vector 검색은 8/12에 wrong hit 8건이 발생해, **exact-first 정책 + 명시적 glossary post-edit**를 최종 채택함.

3. 평가 데이터셋 구성과 EDA

lemon-mint/korean_parallel_sentences_v1.1(49만 한영 병렬, MIT)의 software 도메인 문장에서 한국어 source만으로 ID를 먼저 고정된 뒤 영어 reference를 결합하는 source-only 방식으로 40문장을 선정함. Reference는 glossary 작성에 사용하지 않았고, 사람 검수 correction 6건도 원 reference-provenance와 함께 보존함.

항목	최종 평가셋
규모·도메인	40문장, software 단일 도메인 (과제 요구 30~50문장 충족)
핵심 용어	8개(요구 5개 이상), 44 occurrence, 복수 용어 문장 4개
원문 길이	최소 31 / 평균 55.9 / 중앙 56 / 최대 86자
Reference	원 데이터 영어 문장 34 + reviewer correction 6 (전 문장 reference 보유)
Dataset SHA-256	cdef83d7a78de9071cb850890c1c408cbf0573e5501a21086026f232501e6650

선정 근거·데이터 누수 방지: ① 파일럿과 최종셋 분리 ② source-only ID 고정 후 reference join ③ NFKC 정규화·중복 제거·seed 42 ④ reference와 무관한 독립 glossary 사용 ⑤ runtime에는 source만 전달 ⑥ reference·수동 label은 번역 종료 후 오프라인 채점에서만 결합함.

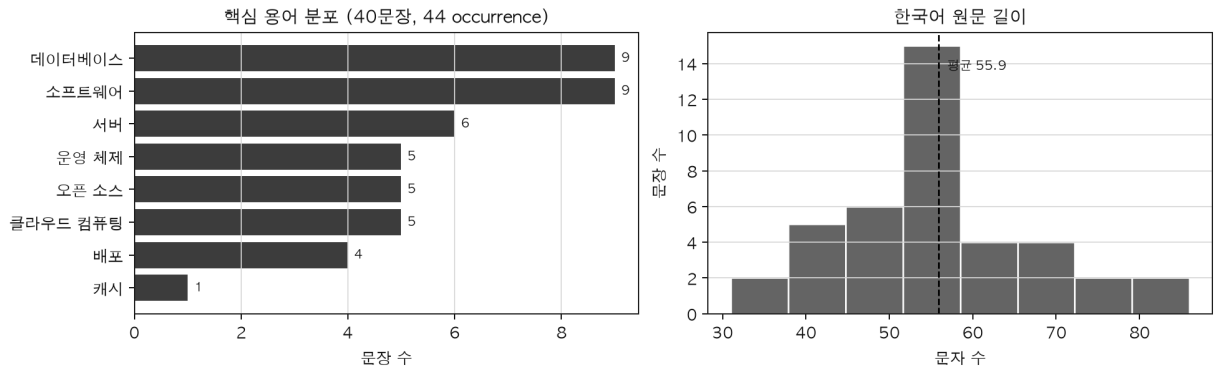


그림 1. 평가셋 EDA — 핵심 용어 분포(좌), 한국어 원문 길이 분포(우)

EDA 해석과 한계: 데이터베이스·소프트웨어가 각 9회로 최다이고 캐시는 1회에 그침. 평균 55.9자의 짧은 software 설명문 중심 표본이므로, 40문장 결과를 다른 도메인·장문·대화 번역으로 일반화할 수 없고 용어별 표본 수도 균일하지 않음.

4. Baseline vs Agent-RAG 벤치마크 결과

동결 run 429d6c4a-a1b6-4514-bfd3-dab6966c4101에서 동일 40문장을 4개 조건으로 실행한 160개 결과임. Latency는 warm-up 적용 단일 동결 run의 관찰값이며 반복 측정 기반 SLA 아님.

조건	용어 정확도	평균 / 중앙 / p95 latency (ms)	문장 수정률	retry 분포
Baseline	40/44 = 90.9%	442.65 / 412.81 / 628.21	N/A	N/A
RAG	44/44 = 100.0%	458.20 / 425.49 / 641.16	N/A	N/A
Agent	41/44 = 93.2%	11,833.72 / 9,392.60 / 21,943.63	9/40 = 22.5%	0회 31 · 1회 9
Agent+RAG	44/44 = 100.0%	11,187.07 / 9,285.75 / 22,367.25	6/40 = 15.0%	0회 34 · 1회 6

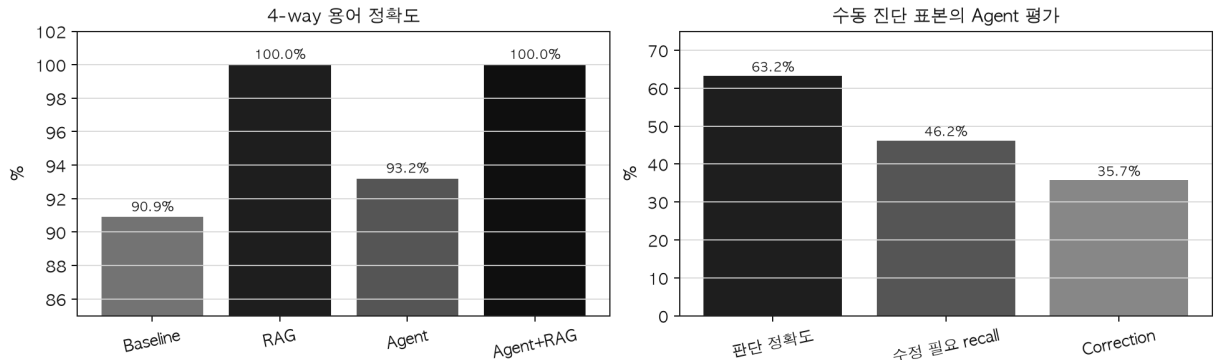


그림 2. 4-way 용어 정확도(좌), 수동 진단 표본의 Agent 평가 지표(우)

RAG는 평균 약 15.55ms(+3.5%)만 추가하며 marked term occurrence 4개를 개선함(90.9%→100%). 단, 용어 정확도 100%는 glossary 용어 반영만 뜻하며 문장 전체 의미 정확도가 아님. Agent 경로는 조건부 retry를 실제 수행했으나 평균 약 11초로 Baseline 대비 약 25~27배 느림.

Agent 품질 판단 정확도(수동 검증 대비): 수동 검수는 무작위 전수 평가가 아니라 retry·모드 불일치·초기 PASS 사례를 의도적으로 고른 10 source × 2 mode = 20 mode-case 진단 표본임. Positive class는 사람이 NEEDS_REVISION으로 판정한 경우임.

범위	TP/TN/FP/FN	판단 정확도	수정 필요 recall	Successful correction	Component failure
전체	6/6/0/7	63.2%	46.2%	5/14 = 35.7%	1/20
Agent	4/3/0/3	70.0%	57.1%	4/7 = 57.1%	0/10
Agent+RAG	2/3/0/4	55.6%	33.3%	1/7 = 14.3%	1/10

오류 유형별 분류·빈도: 수정 필요 14행의 primary error는 누락/추가(omission/addition) 10건, 용어 오역(term) 4건임. False PASS(FN) 7건은 모두 MAJOR였고 그중 6건이 주어·핵심 정보 누락 중심임. Component failure 1건은 판단 정확도 분모에서 제외하되 결과에는 그대로 보고함.

5. 성공·실패 사례와 오류 유형별 근본 원인 분석

구분	사례	관찰
성공	009::agent	오역 Road balance-single-point disorders를 load balancing 계열 표현으로 교정함
성공	019::agent	누락된 Website development 주어와 deployment 정보를 복원함
성공	004::agent	Kubernetes-automate-scaling 표현을 개선함, 일부 term 오류는 잔존함
실패	009::agent_rag	summary 텍스트는 오류를 지적했으나 판정은 PASS, retry 0회로 종료함(판정 모순)
실패	027::agent_rag	revision이 고유명사 Consul을 잘못된 Consult로 변경함(revision regression)
실패	024::agent_rag	계약에 없는 pronoun error type을 출력해 component failure 발생함

근본 원인: ① 원문에만 존재하는 standalone 주어·고유명사 누락을 Judge가 탐지하지 못함 ② 자유 서술 summary와 구조화 decision의 모순 ③ glossary 충족을 문장 품질로 과신 ④ 수정 과정에서 새 오류를 만드는 revision regression. 단, 행동 선정 진단 표본이므로 Agent+RAG가 일반적으로 더 나쁘다는 결론은 내릴 수 없음.

6. 현재 구조의 한계와 개선 아이디어

한계	개선 아이디어(다음 최소 실험)
Judge의 낮은 수정 필요 recall(46.2%)	주어·고유명사·핵심 기술어를 모든 attempt에서 검사하고 근거 부족 시 UNVERIFIABLE로 처리함
판정 필드 모순·JSON 불안정	구조화 error list 기반으로 PASS/REVISE를 코드가 결정하고 malformed 출력은 격리함
Revision regression	수정 전후 entity·숫자·용어 coverage 비교 후 악화 시 안전 rollback 수행함
번역 전 1회 검색으로 고정된 단발성 RAG	ReAct 기반 agentic RAG 도입 — Agent가 추론(Thought)→검색(Action)→관찰(Observation) 루프 로 부족한 용어 근거를 필요한 시점에 재검색해 wrong hit·미반영 용어를 줄이고 정확도를 개선함. 소형 모델의 tool-call 안정성과 latency 증가는 고정 replay에서 먼저 검증함
Few-shot 예시 추가 시 판정 품질이 오히려 저하됨 — 역효과 원인 미규명 상태로 현재 코드에서는 제거 함	동일 입력 고정 replay에서 0/1/3-shot을 비교하고, 실패 출력을 예시 내용 복사·JSON 형식 이탈·판정 뒤집힘으로 유형화해 소형 모델의 few-shot 역효과 원인을 규명한 뒤 재도입 여부를 결정함
작은 단일 도메인 표본(40문장)	저장된 FN 7행·TN 6행 replay 검증 → reserve 수동 검수 → 새 동결 benchmark로 확장함
Agent 경로 약 11초 latency	Judge call 최소화, 더 큰 모델은 동일 입력에서 품질·latency를 함께 비교함

7. 재현 근거와 결론

- API 계약 테스트 18 passed · 실행 .venv/bin/uvicorn translation_qa.main:app --reload
- Dataset SHA-256 cdef83d7...501e6650 · Benchmark artifact SHA-256 e0f858ca...c319590cf
- 근거 파일: data/evaluation_v1.jsonl · docs/EVALUATION_CONTRACT.md · docs/EXPERIMENT_LOG.md · data/manual_reviews/

현재 구현은 용어 통제와 조건부 retry가 동작하는 재현 가능한 MVP임. Exact-first RAG는 동결셋의 용어 정확도를 개선했으나, 사람 검수에서는 Judge recall과 latency가 병목으로 확인됨. 따라서 완벽한 번역 시스템이나 Agent+RAG의 일반적 우위를 주장하지 않으며, 다음 단계는 multi-agent 확장보
다 source coverage 검사와 판정 일관성을 작은 고정 replay에서 먼저 검증하는 것이 적절함.